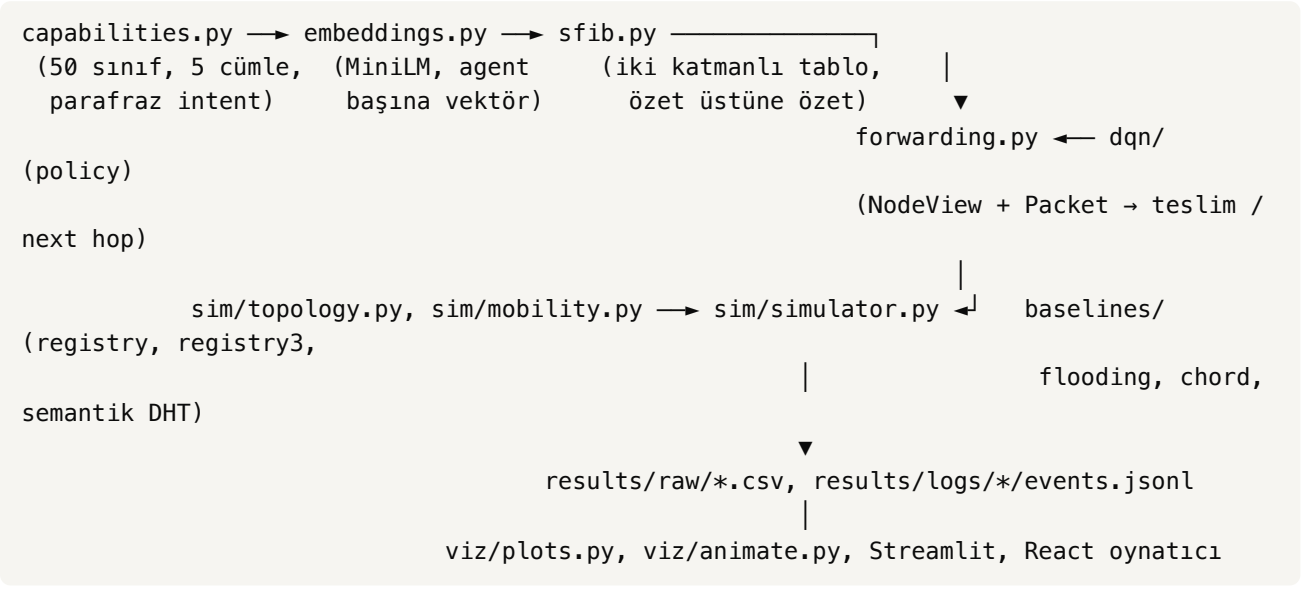
AgentRoute: Nasıl Yaptık

Bu belge her parçanın nasıl tasarlandığını, hangi kararların neden alındığını ve yolda karşılaşılan sorunları anlatır. Kim neden sorumluydu sorusunun cevabı `KIM_NE_YAPTI.md` dosyasında. Sunumdan önce üç kişinin de bu belgenin tamamını okuması gerekir.

Belge v2 halini anlatır. v1'den v2'ye neyin neden değiştiği 7. bölümde. v1 kodu git'te `v1` etiketiyle duruyor.

1. Büyük resim



İki ilke bütün tasarımı belirledi:

1. **Forwarding simülatörden bağımsız.** `forwarding.py` sadece bir düğümün yerel durumunu (`NodeView`) ve paketi görür, yerel bir agent ya da bir komşu döndürür. Simülatör bu fonksiyonu her hop'ta çağırır. Aynı kod ileride bir UDP overlay'e taşınabilir.
2. **Simülasyon ve görselleştirme ayrı.** Simülatör her olayı JSONL'e yazar. Grafikler, animasyon, Streamlit ve React oynatıcı sadece bu dosyaları okur.

2. Adresleme ve S-FIB (Bekir Emre)

2.1 Dataset

- 50 yetenek sınıfı, altı grupta: görüntü, dil, ses, veri/analitik, mobilite-IoT-ağ ve diğer servisler.
- Her sınıfta:
  - **2 yetenek açıklaması.** İki farklı sağlayıcının aynı yeteneği farklı kelimelerle anlatmasını temsil ediyor.
  - **3 örnek istek.** A2A agent kartlarındaki `examples` alanı gibi.
  - **10 parafraz intent.** Kullanıcının gerçekten yazacağı türden günlük cümleler.
- Intent'ler sınıf başına sabit seed ile 7 train / 3 test olarak bölünür (350 / 150). DQN ve τ sadece train ile ayarlanır, bütün deneyler test intent'leriyle yapılır.
- `tests/test_core.py` her intent'in hiçbir kart cümlesini içermediğini ve hiçbir kart cümlesinin intent'i içermediğini kontrol ediyor.

2.2 Capability address = agent başına kendi vektörü

- Sınıfın 5 cümlesi MiniLM ile gömülür (384 boyut, normalize).
- Her agent ağırlıkları Dirichlet(2)'den çeker. Adres = 5 vektörün ağırlıklı toplamı, normalize. Kod: `embeddings.agent_vectors()`.
- **Neden:** v1'de aynı karttaki agent'ların vektörü birebir aynıydı. Bu yüzden tablo "agent" değil "kart" sayıyordu ve özetleme sorunu hiç ortaya çıkmıyordu. v2'de iki sağlayıcı yakın ama aynı değil.

Encoder seçimi. Altı encoder karşılaştırıldı (`experiments/compare_encoders.py`):

Encoder	Boyut	Train top-1	Test top-1
all-MiniLM-L6-v2	384	0.763	0.773
multi-qa-MiniLM-L6-cos-v1	384	0.737	0.733
all-mpnet-base-v2	768	0.840	0.747
bge-small-en-v1.5	384	0.831	0.773
bge-base-en-v1.5	768	0.826	0.773
e5-small-v2	384	0.754	0.740

n = 150 test isteğinde ±7 puandan küçük fark gürültü. Daha büyük encoder train'de iyi, test'te aynı. Üzerine eğitilen küçük bir projeksiyon katmanı (`compare_projection.py`) gördüğü sınıflarda 0.78–0.83, görmediklerinde 0.77–0.81 verdi. Benimsemedik. **Encoder değiştirmek çözüm değil.** (Eleştirel değerlendirmede "daha iyi encoder en büyük kaldıraç" tahmini vardı, ölçüm bunu doğrulamadı.)

2.3 Kabul eşiği τ

- Agent bir işi cos(intent, adres) ≥ τ ise kabul edebilir.
- **v2 ölçütü:** ağın gerçek sorusu "gördüğüm en iyi agent yeterince iyi mi?". Her eğitim isteği için: en iyi agent doğruysa ve τ'yu geçiyorsa +1, yanlışsa ve geçiyorsa −1, kimse geçmiyorsa 0. Yetenek %90 olasılıkla ağda var, yoksa kabul edilen her agent yanlış sayılır. Bu skoru en büyük yapan **τ = 0.355**.
- Test sonucu: en iyi agent doğru sınıftan %76, doğru + τ'yu geçiyor %70.7, yanlış + τ'yu geçiyor %16.7, yetenek ağda yokken reddetme %49.3.
- **τ bir protokol sabiti değil.** 0.25'ten 0.5'e giderken DQN başarısı %63.8'den %41.9'a düşüyor, precision 0.69'dan 0.91'e çıkıyor. Yanlış teslimin bedeline göre seçilmeli.

2.4 S-FIB

- **İki katman:** her komşu için
  - O komşuya doğrudan bağlı agent'lar için **agent başına satır**,
  - Daha uzaktan öğrenilenler için en fazla **k = 8 temsilci vektör** (hop ve ağırlık ile), R = 2'ye kadar.
- **Duyuru kuralı (gerçek distance-vector):** u router'ı w komşusuna kendi agent'larından ve diğer komşularından duyduklarından hesapladığı özeti gönderir. Hop sayısı +1 olur, her adımda yeniden k-means ile sıkıştırılır. Tabloyu dolduran, global bir hesap değil, gerçekten linkten geçen özetlerdir. Mesaj boyutu agent sayısından bağımsız.
- **k-means:** deterministik başlangıç (en ağır vektör, sonra en uzak nokta), ağırlıklı spherical k-means. Karşılaştırma için `mean`, `medoid` ve `exact` var.
- **Bayt sayımı:** her mesajda 40 B başlık, her satırda 388 B int8 vektör + 8 B hop ve ağırlık.

- **Handover:** eski hücre withdraw, yeni hücre announce. Vektör hop hop yayılır. Router bir temsilciyi 0.01'den fazla oynatmıyorsa mesajı daha ileri göndermez, değişiklik periyodik yenilemeyi bekler. Ölçüm: handover başına 20 mesaj, ağ büyüse de sabit.
- **Periyodik yenileme:** her 10 s'de bütün özetler gerçek durumdan yeniden hesaplanır ve gönderilir, her agent gNB'sine yeniden kayıt olur. Trafiğin büyük kısmı budur.
- **Failure:** komşular çöken router'ı 1 s sonra fark eder, tablolar canlı graf üzerinde yeniden kurulur.

2.5 Forwarding ve kabul

- **Paket:** intent vektörü (int8), TTL (16), önceki hop, ziyaret edilen düğümler, oluşma zamanı.
- **Her router'da sırayla:**
  1. Bağlı agent'lardan cos ≥ τ olan en iyisi aday.
  2. **Kabul (best known):** ziyaret edilmemiş bir komşunun duyurduğu aday, exp(–ω·hop) ile zayıflatıldıktan sonra bile bu agent'tan iyiyse teslim edilmez, ω = 0.08 (SPFR'nin değeri).
  3. Policy bir sonraki hop'u seçer. Paket asla geldiği komşuya geri gönderilmez (çıkmaz sokak hariç).
  4. TTL bittiyse: yerel bir agent τ'yu geçiyorsa ona teslim, yoksa başarısız.
- **Greedy:** en yüksek S-FIB skorlu ziyaret edilmemiş komşu. Girdi yoksa default route.
- **Yük duyarlı greedy:** en iyi skorun 0.05 yakınındaki komşulardan en az yüklü olanı. DQN'in yenmesi gereken elle yazılmış kural.
- **SPFR tarzı:** tam tablo, her görünür agent için ϕ = benzerlik · exp(–ω·hop), τ ile kapılı, baskın olana bir hop.
- **Özellikler:** `neighbor_features()` her aksiyon için 15 özellik çıkarır:

Özellik	Anlamı
max_sim, mean_sim	intent ile aksiyonun temsilcileri arasındaki en yüksek / ortalama cosine
sim_gap	bu aksiyonun skoru eksi en iyi aksiyonun skoru
above_tau, has_entry	eşik üstü eşleşme var mı, girdi var mı
hops_norm	en iyi temsilcinin hop mesafesi / (R + 1)
queue_occ	komşunun kuyruk doluluğu (hello mesajlarıyla, 100 ms'de bir)
link_quality, link_delay	1 – kayıp olasılığı, normalize gecikme
in_handover	komşu gNB'de son 2 s içinde handover oldu mu
ttl_ratio, visited	kalan TTL oranı, komşu daha önce ziyaret edildi mi
is_core, is_cloud, is_local	aksiyon türü

3. Simülatör ve baseline'lar (Ömer)

3.1 Topoloji

- **gNB'ler:** hücre ızgarası, 500 m aralık, her gNB aynı zamanda bir edge router. Xn linkleri 4-komşuluk.
- **Core:** daha seyrek bir ızgara (~8 hücreye bir core), core'lar kendi aralarında ızgara.
- **Backhaul:** her gNB en yakın core'a bağlı. Hücrelerin yarısı ikinci en yakına da bağlı.
- **Cloud:** iki core'a 20 ms'lik linklerle bağlı.
- **Gecikmeler:** radyo 5 ms, Xn 2 ms, backhaul 5 ms, core 3 ms.
- **Kayıplar:** kablolu %0.05, radyo %0.2, kablolu linklerin %10'u "bozuk" (%3).

- **Ölçekleme:** hücre sayısı agent sayısıyla büyür (20 agent/hücre, en fazla 400 hücre). 1.000 agent'ta 74 router, 10.000 agent'ta 450 router.

3.2 Router modeli

- **İstekler için:** her router bir SimPy `Resource` (servis 0.2 ms, kuyruk limiti 100).
- **Arka plan trafiği:** M/M/1 kuyruğundaki iş sayısı kadar arka plan paketi bekler. $P(N \geq n) = \rho^n$, bekleme Gamma(N, 1 ms). N ≥ 50 ise paket düşer.
- **Hotspot router'lar (%15):** ortalama 5 s'de bir yoğun (ρ = 0.95) ve normal arasında geçer. Yük çarpanı ρ'yu ölçekler (en fazla 0.97).

3.3 Agent'lar ve mobilite

- **Dağılım:** %40 sabit edge, %50 mobil (random waypoint, hız ×[0.5, 1.5], 2 s duraklama), %10 cloud.
- **Yetenek arzı Zipf(1) dağılımlı:** birkaç yetenek çok yaygın, çoğu nadir. İstekler bütün sınıflardan eşit gelir (bkz. 6.2).
- **Handover:** her saniye konumlar güncellenir, %15 histerezis, 50 ms kesinti.
- **Adil karşılaştırma:** topoloji, mobilite, trafik, ağ ve failure için ayrı random stream'ler var. Aynı seed'de her yöntem aynı istek dizisini görür. Bu, 5. bölümdeki eşleştirilmiş karşılaştırmaları mümkün kılıyor.

3.4 Baseline'lar

- **Merkezi registry:** en merkezi core router'da tek sunucu. Arama 1 ms, kayıt 0.05 ms (bir tablo yazımı). Sorgu yolu: istek → sunucu → cevap → görev. Agent taşındıysa `stale_location`.
- **3 replikalı registry:** istek en yakın canlı replikaya gider, her güncelleme diğerlerine eşitlenir, bu da bayt olarak sayılır.
- **Flooding:** TTL 6, her router bir kez işler. τ'yu geçen her agent cevap verir, cevaplar ağdan geri gelir. İstekçi 200 ms toplar, en benzer cevaba görevi gönderir.
- **Chord (exact):** parafraz başka anahtara gider, %0 **tanım gereği**. Rakip değil, referans.
- **Semantik DHT:** aynı Chord halkası, 64 k-means kovalı açık bir katalog. Agent en yakın kovaya, istek en yakın 2 kovaya bakar.
- **SPFR tarzı:** kendi uyarlamamız, orijinal kod değil. Yük ve fiyat simülatörde olmadığı için fayda = benzerlik.

3.5 Olay logu

CLAUDE.md'deki formatta JSONL (`query`, `hop`, `deliver`, `drop`, `handover`, `adv`, `withdraw`, `move`, `load`, `fail`) ve bir `topology.json`. Büyük deneylerde log kapalı, demo ve Streamlit'te açık.

4. Öğrenen karar (Bekir Göktepe)

4.1 Model

- **Paylaşılan Q-ağı:** ağ her aksiyonu ayrı skorlar. Girdi = 15 özellik, çıktı = tek Q-değeri.
- **Aksiyonlar:** komşular + yerel bir agent τ'yu geçiyorsa LOCAL ("burada teslim et"). DQN böylece hem durup durmayacağına hem nereye gideceğine karar veriyor. v1'de teslim kuralı sabitti, DQN'e karar verecek çok az şey kalıyordu.
- **MLP:** 15 → 64 → 64 → 1.

- **GNN varyantı:** her aksiyon 64 boyuta gömülür, ortalama ve maksimum bir bağlam vektörü olur, skor = MLP([aksiyon, bağlam]).

4.2 Reward

Olay	Reward
Her karar	–(0.05 + gecikme / 0.1 s)
Doğru agent'a teslim	+1
Düşme (TTL, kuyruk, link, yanlış agent, handover, ...)	–1

Greedy, yük duyarlı greedy ve SPFR tarzı için de aynı getiri tutuluyor, yani hepsi DQN'in kendi ölçütüyle karşılaştırılabiliyor.

4.3 Eğitim

- **Double DQN:** online ağ seçer, target ağ değerlendirir. Target her 500 adımda kopyalanır.
- **Hiperparametreler:** buffer 50.000, batch 128, her 4 geçişte bir adım, Adam 1e-3, Huber, gradyan kırpma 1.0, γ = 0.98, ε 1.0 → 0.05 ilk %60'ta.
- **Episode:** 20 s, rastgele senaryo: 300–2.000 agent, hız 0–20 m/s, yük 0.5–4, failure 0/0.1/0.2, R 2–4.
- **Veri:** eğitim ve model seçimi sadece train intent'leriyle.
- **Model seçimi:** her 10 episode'da sabit doğrulama senaryolarında ε = 0 ile getiri ölçülür, en iyisi saklanır. En iyi: MLP 289. episode (–0.131), GNN 199. episode (–0.145). Greedy –0.183, yük duyarlı –0.197.
- **Grafik:** eğitim getirisini göstermiyoruz, çünkü ε düştükçe zaten yükselir. Sadece doğrulama gösteriliyor, üzerinde greedy ve yük duyarlı referans çizgileri var.
- **Süre (M2 CPU):** MLP ≈ 35 dk, GNN ≈ 45 dk (paralel).

5. Deneyler ve görselleştirme (yapay zeka desteğiyle)

- **Deneyler:** agent sayısı, hız (500 m ve 150 m hücre), failure, R, yük, istek hızı, τ, S-FIB içeriği, header, kabul kuralı, güncelleme eşiği, flooding penceresi. 10 yöntem × 10 seed, her koşu 30 s simülasyon.
- **Süre:** 7 paralel işçiyle yaklaşık 1 saat.
- **Metrikler:** başarı, eşleşme tavanı, precision, ikisinin de teslim ettiği isteklerde süre, kontrol kB/s ve istek başına kB (ayrı).
- **Eşleştirilmiş karşılaştırma:** `run_matched.py`, aynı istekler üzerinde fark ve 10 seed üzerinden bootstrap %95 güven aralığı.
- **Animasyon:** demo logundaki gerçek olaylardan kuruluyor. Karşılaştırma videosunda "seçilmiş örnek, ortalama değil" yazıyor.
- **Streamlit, React + D3:** sadece loglardan okur, yeni yöntemlerin etiketleri eklendi.

6. Karşılaşılan sorunlar ve çözümler

6.1 MiniLM eşleşmesi zayıftı

Kısa açıklamalarla top-1 doğruluk 0.61'di. Kart cümleleriyle 0.77'ye çıktı. Kalan hatalar birbirine benzeyen sınıflarda. Encoder değiştirmek bunu çözmedi (bkz. 2.2).

6.2 İlk senaryoda yönlendirmenin önemi yoktu

- Her sınıf eşit dağılımlı ve hücre başına 40 agent varken greedy isteklerin çoğunu kendi hücresinde buluyordu.
- Yetenek arzını Zipf(1) yaptık, hücre yoğunluğunu 20 agent'a indirdik.

6.3 Ölçeklenebilirlik

Benzerlik hesapları numpy ile, arka plan trafiği analitik, flooding kopyaları router başına bir kez. 10.000 agent / 450 router'lık bir koşu yönteme göre 7–80 s.

6.4 Model seçimi

Doğrulama ölçütünü başarı yerine getiri yaptık, çünkü RL'in optimize ettiği şey bu.

6.5 Sıkıştırma tasarımını ölçerek seçtik (v2)

Eğitim intent'leriyle, 1.000 agent, greedy:

Tasarım	Başarı	Kontrol (kB/s)
İlk bulduğunu kabul, yalnız özet, k = 8	%50.1	–
Best known kabul, yalnız özet, k = 8	%53.5	–
İki katman, k = 8, R = 3	%62.4	485
İki katman, k = 8, R = 2	**%64.2**	**483**
Tam tablo, R = 2	%66.1	1.761
SPFR tarzı	%66.5	–
Registry	%66.5	43
Flooding	%67.1	–
Tavan	%71.9	–

Daha kısa TTL başarıyı düşürdü, 16 kaldı.

6.6 Arka plan işi yönetimi

Eğitimi `( ... &)` ile başlatınca süreçler oturumdan koptu, izlenemez oldu. Durdurup takipli arka plan işi olarak yeniden başlattık, yaklaşık 10 dakika kaybettik.

7. v1'den v2'ye ne değişti, neden

Eleştirel değerlendirmedeki her madde için:

Sorun (v1)	v2'de yapılan	Sonuç
Aynı karttaki agent'lar aynı vektörü taşıyordu, tablo gerçek özetleme yapmıyordu	Agent başına vektör, özet üstüne özet, bayt sayımı	Özetlemenin gerçek bedeli görünür oldu (tam tablo 3.6 kat)
"Yoldaki ilk uygun agent" kuralı	`best_known` kabul kuralı	Greedy %56.3 → %60.4, precision 0.70 → 0.75
DQN'in karar verecek şeyi azdı	"Burada teslim et" aksiyonu, 15 özellik	DQN greedy'yi +2.5 puan geçiyor (v1'de geçemiyordu)
Rakip olarak sadece düz greedy vardı	Yük duyarlı greedy, SPFR tarzı	DQN her ikisine karşı da eşleştirilmiş olarak ölçülüyor
Registry tek sunucu ve her kayıt 1 ms	3 replika, kayıt 0.05 ms	10.000 agent'ta 202 ms yerine 83 ms
Flooding'in cevap maliyeti ve penceresi sayılmıyordu	Cevaplar sayılıyor, 200 ms pencere, pencere taraması	Flooding'in süresi artık pencereye bağlı olarak okunuyor
"Mesaj sayısı" tek metrikti	Kontrol kB/s ve istek başına kB ayrı, istek hızı taraması	Registry her hızda daha ucuz, flooding ≈3 istek/s üstünde pahalı
Tavan yoktu	Oracle eşleşme tavanı	1.000 agent'ta %69.3, DQN'in kalan payı 6.4 puan
5 seed, güven aralığı yok	10 seed, eşleştirilmiş bootstrap	Tüm iddialar güven aralıklı
Süre yalnız teslim edilenler üzerinden ortalamaydı	İkisinin de teslim ettiği isteklerde süre	"%7–14 daha hızlı" iddiası düştü, fark 0.2–0.5 ms
Eğitim eğrisi ε yüzünden yükseliyordu	Sadece doğrulama, referans çizgileriyle	Daha dürüst eğri
Registry çökmesi ortalamayla raporlanıyordu	Sunucu ayakta / düştü diye koşullu	%59 ve %18
Karşılaştırma videosu ortalama gibi duruyordu	"Seçilmiş örnek" etiketi	
Mobilite kolaydı	150 m hücreli deney, maliyet grafiği	Başarı yine düz, bedel trafikte
Eksik literatür	INS, Siena, pSearch, IETF draft-li eklendi	
Git'te commit yoktu	`v1` etiketi, `v2` dalı	

v1 ile v2 rakamları doğrudan karşılaştırılamaz: τ, agent vektörleri, kabul kuralı, flooding ve registry maliyetleri değişti. Yine de referans için 1.000 agent'ta v1: greedy %50.6 / DQN %49.7, v2: greedy %60.4 / DQN %62.9. Artışın çoğu DQN'den değil, tablo ve kabul kuralından geliyor.

8. Sonuçlar kısaca (10 seed ortalaması)

Ayrıntılar ve grafikler final raporun 5. bölümünde. Rakamların kaynağı `results/tables.md` ve `results/summary.json`.

Soru	Cevap
DQN aynı tablodaki kurallardan iyi mi?	Evet. Greedy'ye karşı +2.5 puan (GA 1.4–3.7), yük duyarlı greedy'ye karşı +3.5 (GA 2.6–4.4). Yük 4 ve 10.000 agent'ta da aynı yönde.
DQN her şeyden iyi mi?	Hayır. SPFR tarzı tam tabloyla −1.2 puan (3.6 kat kontrol trafiği). Flooding −2.5 puan. 3 replikalı registry −1.1 puan. Registry ile eşit.
Aynı tam tabloda?	DQN, SPFR tarzından +2.2 puan (GA 1.4–2.9), ama 5 ms daha yavaş.
Gecikme?	Eşleştirilmiş isteklerde greedy'ye karşı 0.2–0.5 ms kazanç, yük duyarlıya karşı en fazla 1 ms kayıp. Registry 60 ms, flooding 234 ms (pencereye bağlı).
Trafik?	Kontrol: AgentRoute 485 kB/s, registry 43, SPFR tarzı 1.757. İstek başına: AgentRoute 1.9 kB, registry 1.2, flooding 174.
Hangi hızda kim ucuz?	Registry her hızda. Flooding yaklaşık 3 istek/s altında ucuz.
10.000 agent?	SPFR tarzı %68.4, flooding %67.1, GNN %66.9, DQN %66.5, greedy %64.3, 3 replika %63.8, registry %62.3, semantik DHT %49.6, tavan %71.1.
Mobilite?	Başarı hızdan bağımsız, 150 m hücrede bile. Handover başına 20 mesaj. Kontrol trafiği 150 m hücrede 30 m/s'de 445'ten 1.336 kB/s'ye çıkıyor.
Node failure?	%30 çökmede DQN %62.9 → %60.2. Registry sunucu ayaktaysa %59, çöktüyse %18. 3 replika %59.
Yük?	Yük 4'te DQN %61.0, greedy %58.6, flooding %61.7, registry %56.9.
R?	R = 2 yeterli. Üstünde kontrol trafiği düz, greedy biraz kötüleşiyor. R = 1 DQN'in eğitim aralığı dışında: 54 ms, 6.4 hop.
Tablo içeriği?	Tek ortalama %51.0, k = 4 %58.4, k = 8 %60.4, k = 32 %63.6, tam tablo %64.1 (greedy).
Header?	int8 (388 B) bedava, binary (48 B) yaklaşık 11 puan kaybettiriyor.
GNN?	MLP'den +0.9 puan (GA 0.1–1.7) 1.000 agent'ta, diğer iki senaryoda fark anlamlı değil.